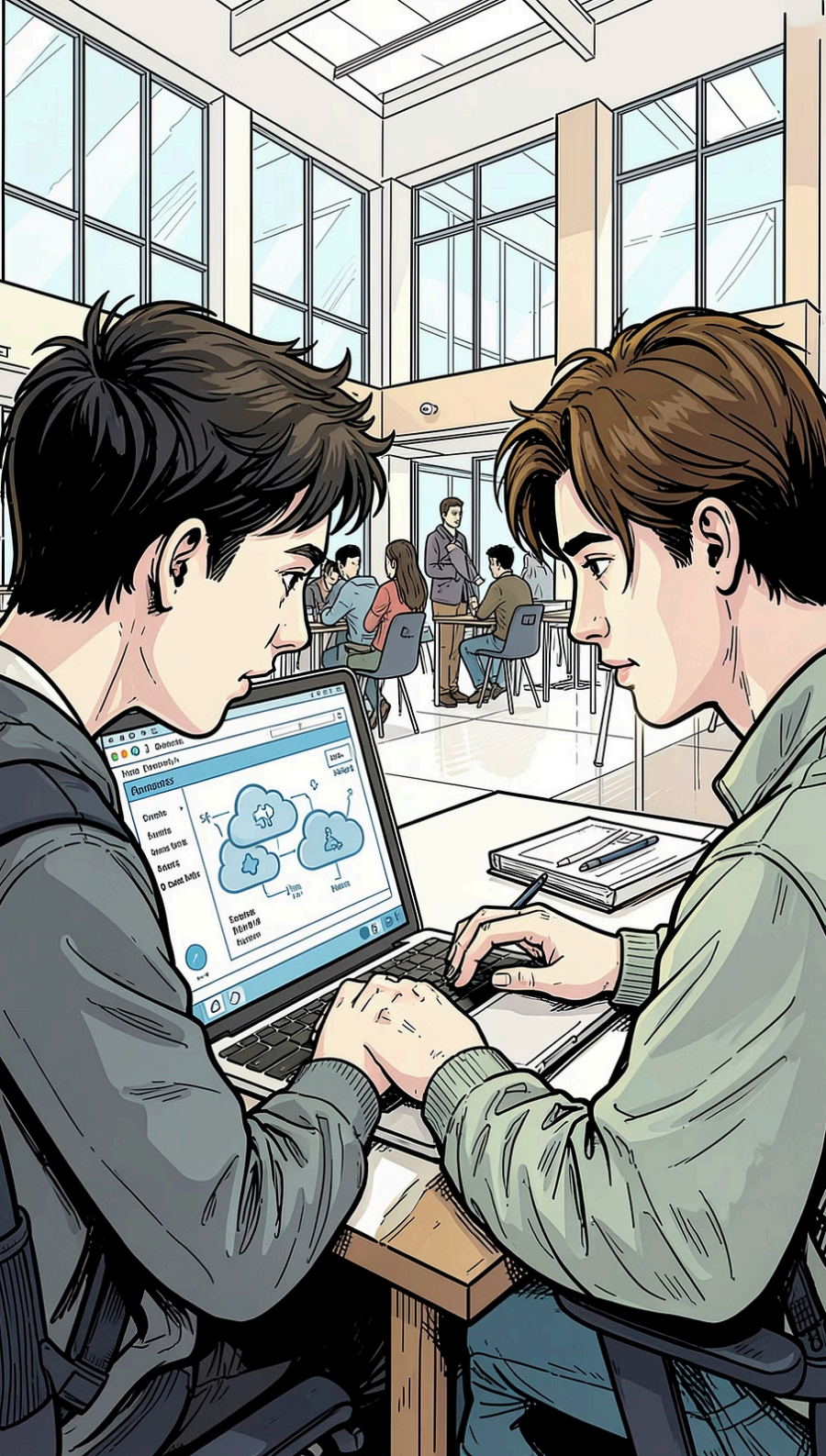






# Azure for Students: Nuvem Real Sem Custo

**US\$ 100 em créditos**

Válidos por 12 meses para explorar serviços Azure

**Sem cartão de crédito**

Validação apenas com e-mail institucional

**Serviços gratuitos**

ACR, ACI, PostgreSQL e mais 25 serviços sempre gratuitos

**Acesso imediato**

Portal real, sem simuladores — ambiente de produção legítimo

# Stack Tecnológica



## Kotlin

Linguagem moderna para JVM, concisa e interoperável com Java



## PostgreSQL

Banco relacional open-source, robusto e amplamente utilizado em produção.



## Azure Container Registry

Registro privado de imagens Docker hospedado na Azure



## Spring Boot

Framework para criar APIs REST com injeção de dependência e segurança



## Docker & Compose

Containerização e orquestração local dos serviços da aplicação



## Azure Container Instances

Execução de containers na nuvem sem gerenciar servidores

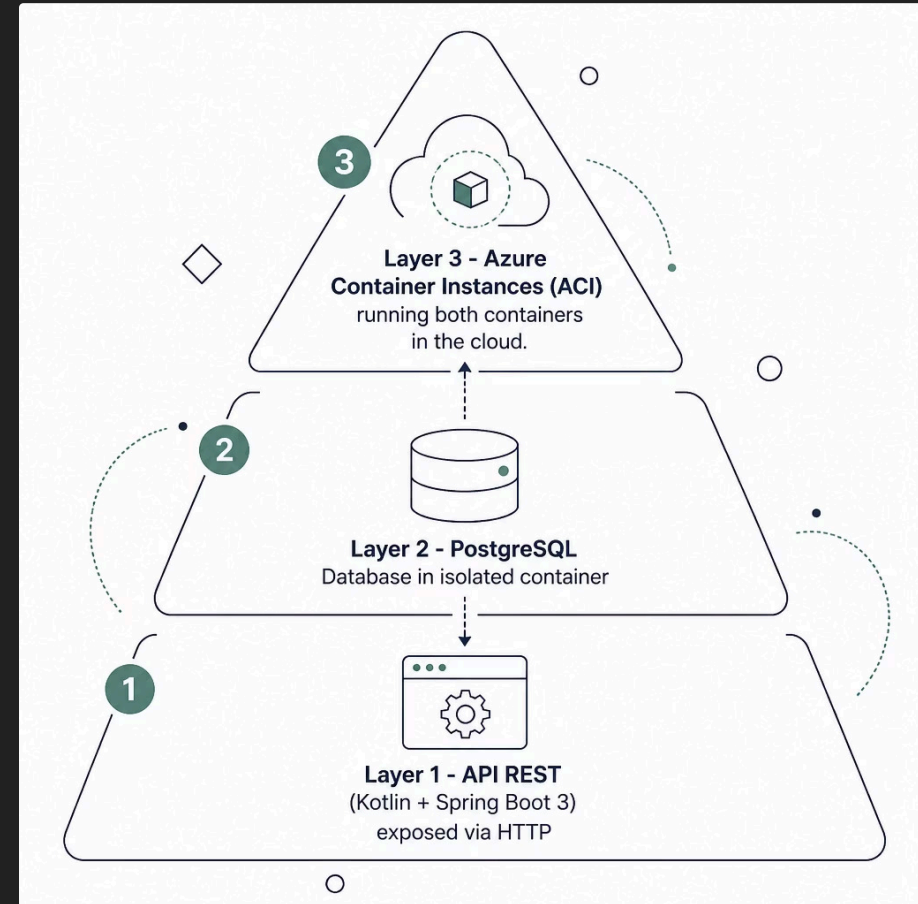


# Visão Geral da Arquitetura

A solução segue uma arquitetura de três camadas totalmente containerizada:

- **API REST:** Kotlin + Spring Boot , exposta via HTTP
- **Banco de dados:** PostgreSQL isolado em container próprio
- **Infraestrutura:** Azure Container Instances gerenciando os containers na nuvem

O Docker Compose orquestra os serviços localmente; o YAML de Container Group replica essa configuração no Azure.



# Containerização com Docker



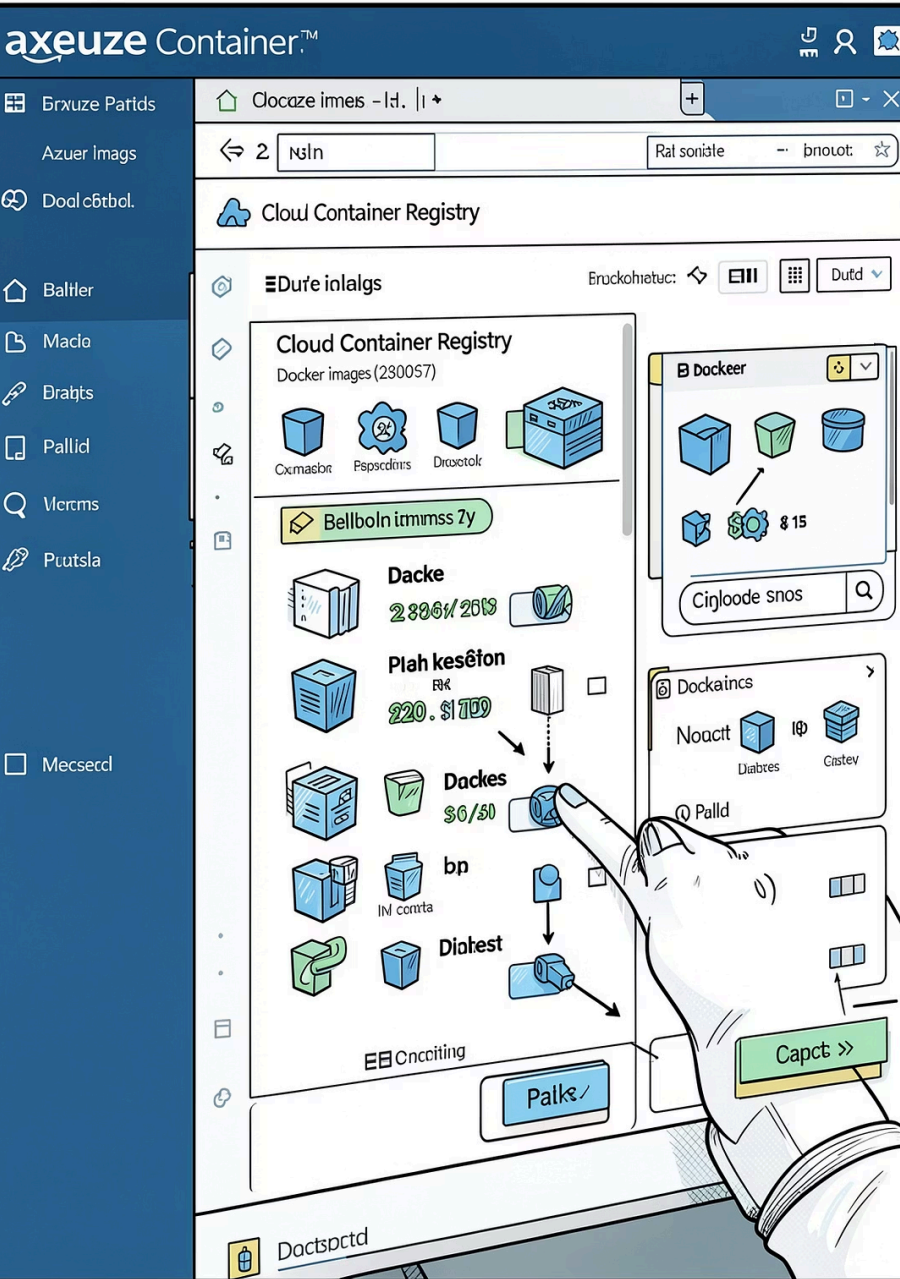
## Dockerfile – API Kotlin

```
FROM eclipse-temurin:17-jdk-alpine
COPY build/libs/api.jar app.jar
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

## docker-compose.yml

```
services:
  api:
    build: .
    ports: ["8080:8080"]
    depends_on: [db]
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD:
        secret
```

O Compose sobe API e banco com um único comando: `docker-compose up`



# Azure Container Registry (ACR)

O ACR armazena as imagens Docker de forma privada e segura, prontas para deploy no Azure.

01

## Criar o registro

Portal Azure → Container Registries  
→ Criar recurso (SKU Basic)

02

## Autenticar no Docker

```
az acr login --name meuRegistro
```

03

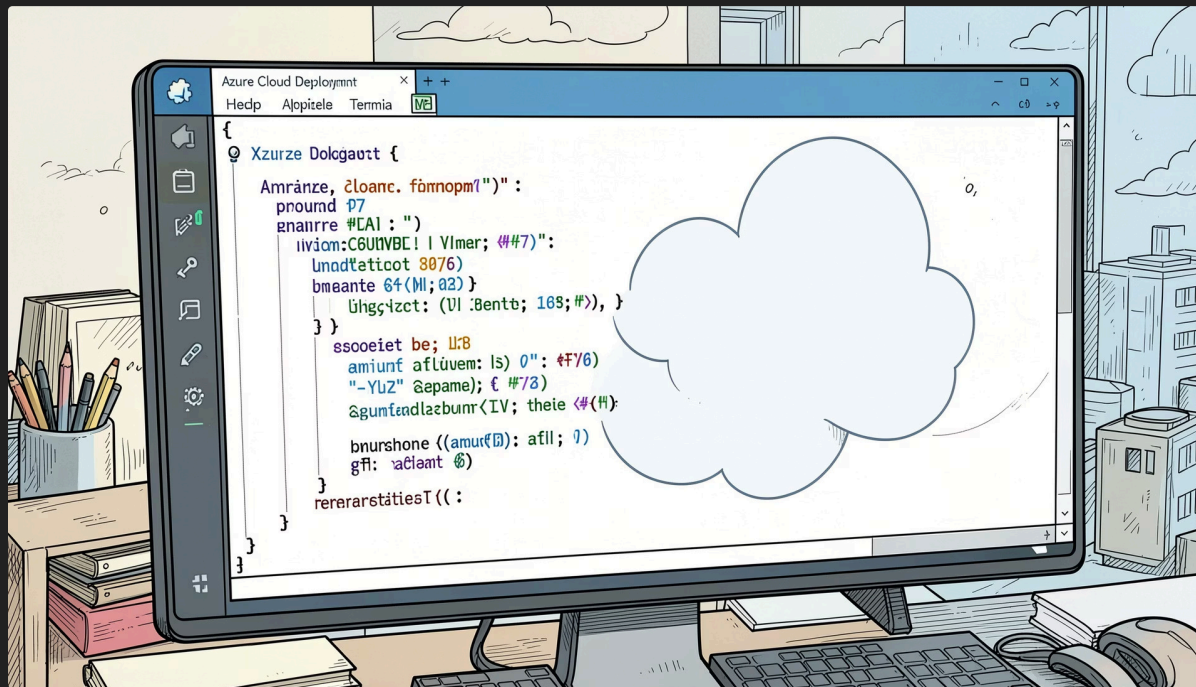
## Build e push da imagem

```
az acr build --registry meuRegistro --image api:v1 .
```

# Deploy na Azure Container Instances

## Container Group via YAML

O arquivo aci.yaml define todos os containers, redes e variáveis de ambiente do grupo:



```
containers:
- name: api
  image:
    meuRegistro.azurecr.io/api:v1
  ports: [{ port: 8080 }]
  environmentVariables:
    - name: DB_HOST
      value: db
    - name: db
      image: postgres:15
      secureEnvironmentVariables:
        - name:
          POSTGRES_PASSWORD
          value: secret
```

Deploy com: `az container create --file aci.yaml`



# Conexão da API com PostgreSQL

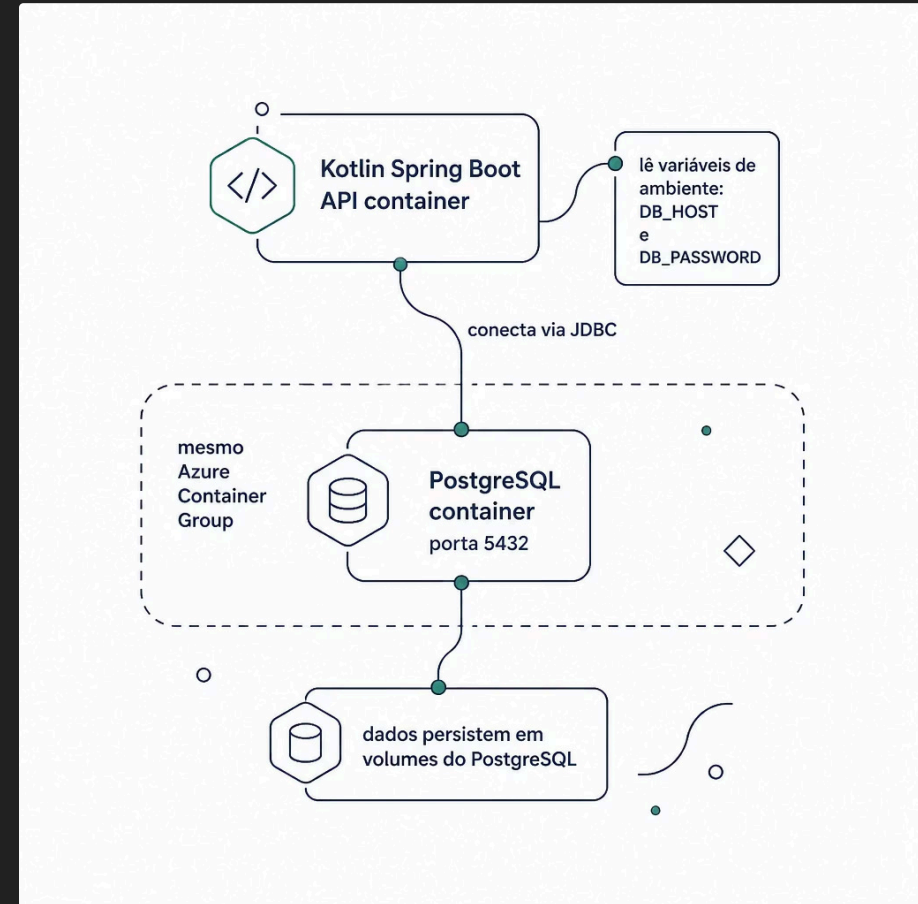
## Configuração no `application.properties`

```
spring.datasource.url=jdbc:postgresql://${DB_HOST:localhost}:5432/mydb
spring.datasource.username=postgres
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=update
```

As credenciais são injetadas via **variáveis de ambiente** no container — sem hardcoded no código.



**Boa prática:** Nunca commitar senhas no repositório. Use sempre variáveis de ambiente ou serviços de segredos como Azure Key Vault.





# Fluxo Completo: Local → ACR → ACI

Desenvolver Local

Build e Push

Implantar no ACI

API Pública

Do código local à API pública na nuvem: todo o ciclo é gerenciado com ferramentas nativas do ecossistema Azure e Docker.

# Desafios, Resultados e Conclusão

## Desafios Encontrados

- Conexão entre containers no ACI: resolvido com variável `DB_HOST=db`
- Autenticação no ACR: resolvido com `az acr login` antes do push.
- IPs de containers ACI são dinâmicos, toda vez que o container reinicia, o IP muda. Isso quebrar a conexão entre API e banco.

## Resultados e Lições

- API acessível publicamente em menos de 30 minutos após o deploy.
- Custo zero com Azure for Students durante todo o projeto.
- Docker Compose + YAML reduzem drasticamente a complexidade operacional.
- Usar `--dns-name-label` para criar um endereço fixo para o banco.

- ✔ **Conclusão:** Usar nuvem real no ambiente acadêmico transforma conceitos teóricos em experiência prática — e o Azure for Students torna isso acessível a qualquer estudante.

# Explore o Código Fonte

Todo o código-fonte e as configurações utilizadas neste projeto estão disponíveis no repositório GitHub:

<https://github.com/younglherme/Cadastro-Produtos-Kotlin>

Sinta-se à vontade para explorar, clonar e adaptar para seus próprios estudos e projetos.

# OBRIGADO!

